Topic 3: Working with JSON Data

Hierarchy, Dot-Notation & Array Indexing in n8n Workflows

1. Document Control

Field	Specification Value
Document Title	Topic 3: Working with JSON Data Specification
Course Module	API Integration Hands-On for n8n (Level 3 - Enterprise Series)
Version	1.0 (Production Master)
Status	**APPROVED & VERIFIED**
Last Updated	2026-11-09
Prepared By	Lead Solutions Architect & Automation Engineering Team
Target Audience	Integration Engineers, API Developers, Automation Trainees

2. Learning Concepts & Architectural Principles

Modern RESTful web services rarely emit flat key-value pairs. Real-world payloads—such as invoices, order requisitions, and customer profiles—are structured hierarchically with:

- **Nested Objects:** Grouping related attributes (e.g., `customer.contact.email`).
- **Homogeneous Collections (Arrays):** Containing multiple line items (e.g., `purchasedItems[0]`).
- **Nullables and Optionals:** Representing conditional fields (`discountCode: null`).

Dot-Notation vs Bracket Indexing in n8n Expressions

n8n leverages JavaScript expression syntax wrapped in double curly braces:

- **Dot-Notation (`.`):** Used to traverse deep object properties: `{{ $json.customer.contact.email }}` (resolves to `"johndoe@example.com"`).
- **0-Based Bracket Indexing (`[0]`):** Used to access specific elements in arrays: `{{ $json.purchasedItems[0].name }}` (resolves to `"Wireless Mouse"`).
- **String Interpolation:** Concatenating variables seamlessly: `{{ $json.customer.firstName }} {{ $json.customer.lastName }}` (resolves to `"John Doe"`).

3. Business Requirements Document (BRD)

ID	Requirement	Business Rationale	Priority
BRD-001	Complex Schema Ingestion	Accept multi-level nested receipt schemas without parsing errors or manual flattening.	HIGH
BRD-002	Precise Hierarchy Traversal	Extract deeply nested contact details via dot notation, preventing silent null pointer exceptions.	CRITICAL
BRD-003	Array Element Isolation	Access specific line items via zero-based array index notation to guarantee item-level accounting.	HIGH
BRD-004	Dynamic Outbound Serialization	Inject dynamic variables into outbound REST notification payloads to automate customer communication.	HIGH
BRD-005	Complete Canvas Testability	Provide pre-pinned test data on every node for 1-click execution across training and CI/CD environments.	MEDIUM

4. Product Requirements Document (PRD)

ID	Product Capability	User Value & Acceptance Criteria	Priority
PRD-001	Nested Mock Data Staging	Set node loading an invoice receipt with customer and item arrays.	High
PRD-002	Dot-Notation Email & Name Extraction	Extracts `customerEmail` and composite `fullName` via clean n8n expressions.	Critical
PRD-003	Array Index Item Access	Fetches the first purchased item's name and price via `[0]` index notation.	High
PRD-004	Dynamic HTTP POST Dispatch	POSTs to JSONPlaceholder with title and body referencing extracted keys; receives HTTP 201.	High
PRD-005	Audit Verification Ledger	Terminal Code node confirming all traversed keys and the outbound 201 response.	High

5. Software Requirements Specification (SRS) & Canvas Blueprint

```
[ N03-01: Manual_Trigger ]
            |
            ▼
[ N03-02: Mock Invoice Data ] ————————> Defines receipt REC-9988 with nested customer & purchasedItems
            |
            ▼
[ N03-03: Extract Customer Data ] ————> Evaluates dot-notation & array indexing:
            |                           - customerEmail: {{ $json.customer.contact.email }}
            |                           - fullName: {{ $json.customer.firstName }} {{ $json.customer.lastNa
            |                           - firstItemName: {{ $json.purchasedItems[0].name }}
            |                           - firstItemPrice: {{ $json.purchasedItems[0].price }}
            ▼
[ N03-04: HTTP_SendNotification ] ————> POST https://jsonplaceholder.typicode.com/posts
            |                           Body: { "title": "New Purchase for John Doe", ... }
            ▼
[ N03-05: Code_AuditSummary ] ————————> Emits terminal validation receipt confirming 100% PASS
```

6. Detailed Node Configuration Blueprint

Node Name	Type & Version	Operational Logic & Configuration
Manual_Trigger	manualTrigger (v1)	Canvas testing trigger for immediate execution on button click.
Mock Invoice Data	set (v3.4)	Mode: Raw JSON. Loads complete receipt REC-9988 schema with customer and item array.
Extract Customer Data	set (v3.4)	Mode: Fields. Maps 4 discrete fields: customerEmail, fullName, firstItemName, firstItemPrice.
HTTP_SendNotification	httpRequest (v4.2)	Method: POST, URL: https://jsonplaceholder.typicode.com/posts, sends dynamic JSON payload.
Code_AuditSummary	code (v2)	Consolidates end-to-end traversal receipt verifying all expressions and outbound API returns.

7. Testing & Quality Assurance Verification Plan

Scenario ID	Test Condition	Expression Evaluated	Expected Result & Verification
TC-03-01	Deep Dot-Notation Email	`{{ $json.customer.contact.email }}`	Resolves to "johndoe@example.com".
TC-03-02	Composite Name String	`{{ $json.customer.firstName }} {{ $json.customer.lastName }}`	Resolves to "John Doe".
TC-03-03	First Array Element Name	`{{ $json.purchasedItems[0].name }}`	Resolves to "Wireless Mouse".
TC-03-04	First Array Element Price	`{{ $json.purchasedItems[0].price }}`	Resolves to numeric 25.99.
TC-03-05	Dynamic Outbound POST	Dynamic Title & Body	Status 201 Created with generated post ID 101.
TC-03-06	1-Click Canvas Run	Click Execute workflow	100% green checkmarks across entire canvas.